van Kampen with selected basepoints: complete alternative author material for canonical 900000088

The Univalent Foundations Program

HoTT/book 578b85cc8d586b1677ec4335148adeb443057d24; CC BY-SA 3.0 Unported

Editorial scope. This is the author's improved computable presentation of the same pushout fundamental-groupoid identification as canonical 900000088. It is retained as alternate author material and does not count as a new problem. The complete naive proof is included because the improved proof explicitly refers to it. Original author compression and symmetric-case omissions remain unchanged.

8.7 The van Kampen theorem

The van Kampen theorem calculates the fundamental group π_1 of a (homotopy) pushout of spaces. It is traditionally stated for a topological space X which is the union of two open subspaces U and V, but in homotopy-theoretic terms this is just a convenient way of ensuring that X is the pushout of U and V over their intersection. Thus, we will prove a version of the van Kampen theorem for arbitrary pushouts.

In this section we will describe a proof of the van Kampen theorem which uses the same encode-decode method that we used for $\pi_1(\mathbb{S}^1)$ in original section 8.1. There is also a more homotopy-theoretic approach; see original ex 9.11.

We need a more refined version of the encode-decode method. In original section 8.1 (as well as in original sections 2.12 and 2.13) we used it to characterize the path space of a (higher) inductive type W — deriving as a consequence a characterization of the loop space $\Omega(W)$, and thereby also of its 0-truncation $\pi_1(W)$. In the van Kampen theorem, our goal is only to characterize the fundamental group $\pi_1(W)$, and we do not have any explicit description of the loop spaces or the path spaces to use.

It turns out that we can use the same technique directly for a truncated version of the path fibration, thereby characterizing not only the fundamental *group* $\pi_1(W)$, but also the whole fundamental *groupoid*. Specifically, for a type X, write $\Pi_1 X : X \to X \to \mathcal{U}$ for the 0-truncation of its identity type, i.e.

$\Pi_1 X(x,y) :\equiv \|x = y\|_0$. Note that we have induced groupoid operations

$$\begin{aligned}(- \cdot -) &: \Pi_1 X(x,y) \to \Pi_1 X(y,z) \to \Pi_1 X(x,z)\\ (-)^{-1} &: \Pi_1 X(x,y) \to \Pi_1 X(y,x)\\ \mathsf{refl}_x &: \Pi_1 X(x,x)\\ \mathsf{ap}_f &: \Pi_1 X(x,y) \to \Pi_1 Y(fx,fy)\end{aligned}$$

for which we use the same notation as the corresponding operations on paths.

8.7.1 Naive van Kampen

We begin with a "naive" version of the van Kampen theorem, which is useful but not quite as useful as the classical version. In §8.7.2 we will improve it to a more useful version.

Given types A, B, C and functions $f : A \to B$ and $g : A \to C$, let P be their pushout $B \sqcup^A C$. As we saw in original section 6.8, P is the higher inductive type generated by

- $i : B \to P$,
- $j : C \to P$, and
- for all $x : A$, a path $kx : ifx = jgx$.

Define $\mathsf{code} : P \to P \to \mathcal{U}$ by double induction on P as follows.

- $\mathsf{code}(ib, ib')$ is a set-quotient (see original section 6.10) of the type of sequences
$$(b, p_0, x_1, q_1, y_1, p_1, x_2, q_2, y_2, p_2, \dots, y_n, p_n, b')$$
where
  - $n : \mathbb{N}$
  - $x_k : A$ and $y_k : A$ for $0 < k \le n$
  - $p_0 : \Pi_1 B(b, fx_1)$ and $p_n : \Pi_1 B(fy_n, b')$ for $n > 0$, and $p_0 : \Pi_1 B(b, b')$ for $n = 0$
  - $p_k : \Pi_1 B(fy_k, fx_{k+1})$ for $1 \le k < n$
  - $q_k : \Pi_1 C(gx_k, gy_k)$ for $1 \le k \le n$

  The quotient is generated by the following equalities:
$$\begin{aligned}(\dots, q_k, y_k, \mathsf{refl}_{fy_k}, y_k, q_{k+1}, \dots) &= (\dots, q_k \cdot q_{k+1}, \dots)\\ (\dots, p_k, x_k, \mathsf{refl}_{gx_k}, x_k, p_{k+1}, \dots) &= (\dots, p_k \cdot p_{k+1}, \dots)\end{aligned}$$
  (see Remark 8.7.3 below). We leave it to the reader to define this type of sequences precisely as an inductive type.
- $\mathsf{code}(jc, jc')$ is identical, with the roles of B and C reversed. We likewise notationally reverse the roles of x and y, and of p and q.
- $\mathsf{code}(ib, jc)$ and $\mathsf{code}(jc, ib)$ are similar, with the parity changed so that they start in one type and end in the other.

- For $a : A$ and $b : B$, we require an equivalence

$$\mathsf{code}(ib, ifa) \simeq \mathsf{code}(ib, jga). \tag{8.7.1}$$

  We define this to consist of the two functions defined on sequences by

$$\begin{aligned}(\dots, y_n, p_n, fa) &\mapsto (\dots, y_n, p_n, a, \mathsf{refl}_{ga}, ga),\\ (\dots, x_n, p_n, a, \mathsf{refl}_{fa}, fa) &\mapsto (\dots, x_n, p_n, ga).\end{aligned}$$

  Both of these functions are easily seen to respect the equivalence relations, and hence to define functions on the types of codes. The left-to-right-to-left composite is

$$(\dots, y_n, p_n, fa) \mapsto (\dots, y_n, p_n, a, \mathsf{refl}_{ga}, a, \mathsf{refl}_{fa}, fa)$$

  which is equal to the identity by a generating equality of the quotient. The other composite is analogous. Thus we have defined an equivalence (8.7.1).
- Similarly, we require equivalences

$$\begin{aligned}\mathsf{code}(jc, ifa) &\simeq \mathsf{code}(jc, jga)\\ \mathsf{code}(ifa, ib) &\simeq (jga, ib)\\ \mathsf{code}(ifa, jc) &\simeq (jga, jc)\end{aligned}$$

  all of which are defined in exactly the same way (the second two by adding reflexivity terms on the beginning rather than the end).
- Finally, we need to know that for $a, a' : A$, the following diagram commutes:

$$\begin{array}{ccc}\mathsf{code}(ifa, ifa') & \longrightarrow & \mathsf{code}(ifa, jga')\\ \downarrow & & \downarrow\\ \mathsf{code}(jga, ifa') & \longrightarrow & \mathsf{code}(jga, jga')\end{array} \tag{8.7.2}$$

  This amounts to saying that if we add something to the beginning and then something to the end of a sequence, we might as well have done it in the other order.

Remark 8.7.3. One might expect to see in the definition of code some additional generating equations for the set-quotient, such as

$$\begin{aligned}(\dots, p_{k-1} \cdot fw, x'_k, q_k, \dots) &= (\dots, p_{k-1}, x_k, gw \cdot q_k, \dots) \quad (\text{for } w : \Pi_1 A(x_k, x'_k))\\ (\dots, q_k \cdot gw, y'_k, p_k, \dots) &= (\dots, q_k, y_k, fw \cdot p_k, \dots). \quad (\text{for } w : \Pi_1 A(y_k, y'_k))\end{aligned}$$

However, these are not necessary! In fact, they follow automatically by path induction on w. This is the main difference between the "naive" van Kampen theorem and the more refined one we will consider in the next subsection.

Continuing on, we can characterize transporting in the fibration code:

- For $p : b =_B b'$ and $u : P$, we have

$$\mathsf{transport}^{b \mapsto \mathsf{code}(u,ib)}(p, (\dots, y_n, p_n, b)) = (\dots, y_n, p_n \cdot p, b').$$

- For $q : c =_C c'$ and $u : P$, we have

$$\mathsf{transport}^{c \mapsto \mathsf{code}(u,jc)}(q, (\dots, x_n, q_n, c)) = (\dots, x_n, q_n \cdot q, c').$$

Here we are abusing notation by using the same name for a path in X and its image in $\Pi_1 X$. Note that transport in $\Pi_1 X$ is also given by concatenation with (the image of) a path. From this we can prove the above statements by induction on u. We also have:

- For $a : A$ and $u : P$,

$$\mathsf{transport}^{v \mapsto \mathsf{code}(u,v)}(ha, (\dots, y_n, p_n, fa)) = (\dots, y_n, p_n, a, \mathsf{refl}_{ga}, ga).$$

This follows essentially from the definition of code.

We also construct a function

$$r : \prod_{u:P} \mathsf{code}(u, u)$$

by induction on u as follows:

$$\begin{aligned} rib &:\equiv (b, \mathsf{refl}_b, b) \\ rjc &:\equiv (c, \mathsf{refl}_c, c) \end{aligned}$$

and for rka we take the composite equality

$$\begin{aligned} (ka, ka)_*(fa, \mathsf{refl}_{fa}, fa) &= (ga, \mathsf{refl}_{ga}, a, \mathsf{refl}_{fa}, a, \mathsf{refl}_{ga}, ga) \\ &= (ga, \mathsf{refl}_{ga}, ga) \end{aligned}$$

where the first equality is by the observation above about transporting in code, and the second is an instance of the set quotient relation used to define code.

We will now prove:

Theorem 8.7.4 (Naive van Kampen theorem)**.** *For all $u, v : P$ there is an equivalence*

$$\Pi_1 P(u, v) \simeq \mathsf{code}(u, v).$$

Proof. To define a function

$$\mathsf{encode} : \Pi_1 P(u, v) \to \mathsf{code}(u, v)$$

it suffices to define a function $(u =_P v) \to \mathsf{code}(u, v)$, since $\mathsf{code}(u, v)$ is a set. We do this by transport:

$$\mathsf{encode}(p) :\equiv \mathsf{transport}^{v \mapsto \mathsf{code}(u,v)}(p, r(u)).$$

Now to define

$$\mathsf{decode} : \mathsf{code}(u, v) \to \Pi_1 P(u, v)$$

we proceed as usual by induction on $u, v : P$. In each case for u and v, we apply i or j to all the equalities p_k and q_k as appropriate and concatenate the results in P, using h to identify the endpoints. For instance, when $u \equiv ib$ and $v \equiv ib'$, we define

$$\mathsf{decode}(b, p_0, x_1, q_1, y_1, p_1, \dots, y_n, p_n, b') :\equiv (p_0) \cdot h(x_1) \cdot j(q_1) \cdot h(y_1)^{-1} \cdot i(p_1) \cdot \dots \cdot h(y_n)^{-1} \cdot i(p_n). \tag{8.7.5}$$

This respects the set-quotient equivalence relation and the equivalences such as (8.7.1), since $h : fi \sim gj$ is natural and f and g are functorial.

As usual, to show that the composite

$$\Pi_1 P(u,v) \xrightarrow{\mathsf{encode}} \mathsf{code}(u,v) \xrightarrow{\mathsf{decode}} \Pi_1 P(u,v)$$

is the identity, we first peel off the 0-truncation (since the codomain is a set) and then apply path induction. The input refl_u goes to ru, which then goes back to refl_u (applying a further induction on u to decompose $\mathsf{decode}(ru)$).

Finally, consider the composite

$$\mathsf{code}(u,v) \xrightarrow{\mathsf{decode}} \Pi_1 P(u,v) \xrightarrow{\mathsf{encode}} \mathsf{code}(u,v).$$

We proceed by induction on $u, v : P$. When $u \equiv ib$ and $v \equiv ib'$, this composite is

$$\begin{aligned}
(b, p_0, x_1, q_1, y_1, p_1, \dots, y_n, p_n, b') &\mapsto \left(ip_0 \cdot hx_1 \cdot jq_1 \cdot hy_1{}^{-1} \cdot ip_1 \cdot \dots \cdot hy_n{}^{-1} \cdot ip_n\right)_* (rib) \\
&= (ip_n)_* \cdots (jq_1)_* (hx_1)_* (ip_0)_* (b, \mathsf{refl}_b, b) \\
&= (ip_n)_* \cdots (jq_1)_* (hx_1)_* (b, p_0, if x_1) \\
&= (ip_n)_* \cdots (jq_1)_* (b, p_0, x_1, \mathsf{refl}_{gx_1}, jgx_1) \\
&= (ip_n)_* \cdots (b, p_0, x_1, q_1, jgy_1) \\
&= \quad \vdots \\
&= (b, p_0, x_1, q_1, y_1, p_1, \dots, y_n, p_n, b').
\end{aligned}$$

i.e., the identity function. (To be precise, there is an implicit inductive argument needed here.) The other three point cases are analogous, and the path cases are trivial since all the types are sets. □

Theorem 8.7.4 allows us to calculate the fundamental groups of many types, provided A is a set, for in that case, each $\mathsf{code}(u,v)$ is, by definition, a set-quotient of a *set* by a relation.

Example 8.7.6. Let $A :\equiv \mathbf{2}$, $B :\equiv \mathbf{1}$, and $C :\equiv \mathbf{1}$. Then $P \simeq S^1$. Inspecting the definition of, say, $\mathsf{code}(i(\star), i(\star))$, we see that the paths all may as well be trivial, so the only information is in the sequence of elements $x_1, y_1, \dots, x_n, y_n : \mathbf{2}$. Moreover, if we have $x_k = y_k$ or $y_k = x_{k+1}$ for any k, then the set-quotient relations allow us to excise both of those elements. Thus, every such sequence is equal to a canonical *reduced* one in which no two adjacent elements are equal. Clearly such a reduced sequence is uniquely determined by its length (a natural number n) together with, if $n > 1$, the information of whether x_1 is $0_{\mathbf{2}}$ or $1_{\mathbf{2}}$, since that determines the rest of the sequence uniquely. And these data can, of course, be identified with an integer, where n is the absolute value and x_1 encodes the sign. Thus we recover $\pi_1(S^1) \cong \mathbb{Z}$.

Since Theorem 8.7.4 asserts only a bijection of families of sets, this isomorphism $\pi_1(S^1) \cong \mathbb{Z}$ is likewise only a bijection of sets. We could, however, define a concatenation operation on code (by concatenating sequences) and show that encode and decode form an isomorphism respecting this structure. (In the language of original chapter 9, these would be "pregroupoids".) We leave the details to the reader.

Example 8.7.7. More generally, let $B :\equiv \mathbf{1}$ and $C :\equiv \mathbf{1}$ but A be arbitrary, so that P is the suspension of A. Then once again the paths p_k and q_k are trivial, so that the only information in a path code is a sequence of elements $x_1, y_1, \dots, x_n, y_n : A$. The first two generating equalities say that adjacent equal elements can be canceled, so it makes sense to think of this sequence as a word of the form

$$x_1 y_1^{-1} x_2 y_2^{-1} \cdots x_n y_n^{-1}$$

in a group. Indeed, it looks similar to the free group on A (or equivalently on $\|A\|_0$; see original rmk 6.11.8), but we are considering only words that start with a non-inverted element, alternate between inverted and non-inverted elements, and end with an inverted one. This effectively reduces the size of the generating set by one. For instance, if A has a point $a : A$, then we can identify $\pi_1(\Sigma A)$ with the group presented by $\|A\|_0$ as generators with the relation $|a|_0 = e$; see original exs 8.10 and 8.11 for details.

Example 8.7.8. Let $A :\equiv \mathbf{1}$ and B and C be arbitrary, so that f and g simply equip B and C with basepoints b and c, say. Then P is the *wedge* $B \vee C$ of B and C (the coproduct in the category of based spaces). In this case, it is the elements x_k and y_k which are trivial, so that the only information is a sequence of loops $(p_0, q_1, p_1, \dots, p_n)$ with $p_k : \pi_1(B, b)$ and $q_k : \pi_1(C, c)$. Such sequences, modulo the equivalence relation we have imposed, are easily identified with the explicit description of the *free product* of the groups $\pi_1(B, b)$ and $\pi_1(C, c)$, as constructed in original section 6.11. Thus, we have $\pi_1(B \vee C) \cong \pi_1(B) * \pi_1(C)$.

However, Theorem 8.7.4 stops just short of being the full classical van Kampen theorem, which handles the case where A is not necessarily a set, and states that $\pi_1(B \sqcup^A C) \cong \pi_1(B) *_{\pi_1(A)} \pi_1(C)$ (with base point coming from A). Indeed, the conclusion of Theorem 8.7.4 says nothing at all about $\pi_1(A)$; the paths in A are "built into the quotienting" in a type-theoretic way that makes it hard to extract explicit information, in that $\mathsf{code}(u, v)$ is a set-quotient of a non-set by a relation. For this reason, in the next subsection we consider a better version of the van Kampen theorem.

8.7.2 The van Kampen theorem with a set of basepoints

The improvement of van Kampen we present now is closely analogous to a similar improvement in classical algebraic topology, where A is equipped with a *set S of base points*. In fact, it turns out to be unnecessary for our proof to assume that the "set of basepoints" is a *set* — it might just as well be an arbitrary type; the utility of assuming S is a set arises later, when applying the theorem to obtain computations. What is important is that S contains at least one point in each connected component of A. We state this in type theory by saying that we have a type S and a function $k : S \to A$ which is surjective, i.e. (-1)-connected. If $S \equiv A$ and k is the identity function, then we will recover the naive van Kampen theorem. Another example to keep in mind is when A is pointed and (0-)connected, with $k : \mathbf{1} \to A$ the point: by original lems 7.5.2 and 7.5.11 this map is surjective just when A is 0-connected.

Let $A, B, C, f, g, P, i, j, h$ be as in the previous section. We now define, given our surjective map $k : S \to A$, an auxiliary type which improves the connectedness of k. Let T be the higher inductive type generated by

- A function $\ell : S \to T$, and
- For each $s, s' : S$, a function $m : (ks =_A ks') \to (\ell s =_T \ell s')$.

There is an obvious induced function $\overline{k} : T \to A$ such that $\overline{k}\ell = k$, and any $p : ks = ks'$ is equal to the composite $ks = \overline{k}\ell s \overset{\overline{k}mp}{=} \overline{k}\ell s' = ks'$.

Lemma 8.7.9. *$\overline{k}$ is 0-connected.*

Proof. We must show that for all $a : A$, the 0-truncation of the type $\sum_{(t:T)}(\overline{k}t = a)$ is contractible. Since contractibility is a mere proposition and k is (-1)-connected, we may assume that $a = ks$ for some $s : S$. Now we can take the center of contraction to be $|(\ell s, q)|_0$ where q is the equality $\overline{k}\ell s = ks$.

It remains to show that for any $\phi : \left\|\sum_{(t:T)}(\overline{k}t = ks)\right\|_0$ we have $\phi = |(\ell s, q)|_0$. Since the latter is a mere proposition, and in particular a set, we may assume that $\phi = |(t, p)|_0$ for $t : T$ and $p : \overline{k}t = ks$.

Now we can do induction on $t : T$. If $t \equiv \ell s'$, then $ks' = \overline{k}\ell s' \overset{p}{=} ks$ yields via m an equality $\ell s = \ell s'$. Hence by definition of $\overline{k}$ and of equality in homotopy fibers, we obtain an equality $(ks', p) = (ks, q)$, and thus $|(ks', p)|_0 = |(ks, q)|_0$. Next we must show that as t varies along m these equalities agree. But they are equalities in a set (namely $\left\|\sum_{(t:T)}(\overline{k}t = ks)\right\|_0$), and hence this is automatic. □

Remark 8.7.10. T can be regarded as the (homotopy) coequalizer of the "kernel pair" of k. If S and A were sets, then the (-1)-connectivity of k would imply that A is the 0-truncation of this coequalizer (see original chapter 10). For general types, higher topos theory suggests that (-1)-connectivity of k will imply instead that A is the colimit (a.k.a. "geometric realization") of the "simplicial kernel" of k. The type T is the colimit of the "1-skeleton" of this simplicial kernel, so it makes sense that it improves the connectivity of k by 1. More generally, we might expect the colimit of the n-skeleton to improve connectivity by n.

Now we define $\mathsf{code} : P \to P \to \mathcal{U}$ by double induction as follows

- $\mathsf{code}(ib, ib')$ is now a set-quotient of the type of sequences

$$(b, p_0, x_1, q_1, y_1, p_1, x_2, q_2, y_2, p_2, \dots, y_n, p_n, b')$$

  where

  - $n : \mathbb{N}$,
  - $x_k : S$ and $y_k : S$ for $0 < k \leq n$,
  - $p_0 : \Pi_1 B(b, fkx_1)$ and $p_n : \Pi_1 B(fky_n, b')$ for $n > 0$, and $p_0 : \Pi_1 B(b, b')$ for $n = 0$,
  - $p_k : \Pi_1 B(fky_k, fkx_{k+1})$ for $1 \leq k < n$,
  - $q_k : \Pi_1 C(gkx_k, gky_k)$ for $1 \leq k \leq n$.

The quotient is generated by the following equalities (see Remark 8.7.3):

$$\begin{aligned}
(\dots, q_k, y_k, \mathsf{refl}_{fy_k}, y_k, q_{k+1}, \dots) &= (\dots, q_k \centerdot q_{k+1}, \dots) \\
(\dots, p_k, x_k, \mathsf{refl}_{gx_k}, x_k, p_{k+1}, \dots) &= (\dots, p_k \centerdot p_{k+1}, \dots) \\
(\dots, p_{k-1} \centerdot fw, x'_k, q_k, \dots) &= (\dots, p_{k-1}, x_k, gw \centerdot q_k, \dots) \\
&\qquad (\text{for } w : \Pi_1 A(kx_k, kx'_k)) \\
(\dots, q_k \centerdot gw, y'_k, p_k, \dots) &= (\dots, q_k, y_k, fw \centerdot p_k, \dots). \\
&\qquad (\text{for } w : \Pi_1 A(ky_k, ky'_k))
\end{aligned}$$

We will need below the definition of the case of decode on such a sequence, which as before concatenates all the paths p_k and q_k together with instances of h to give an element of $\Pi_1 P(ifb, ifb')$, cf. (8.7.5). As before, the other three point cases are nearly identical.

- For $a : A$ and $b : B$, we require an equivalence

$$\mathsf{code}(ib, ifa) \simeq \mathsf{code}(ib, jga). \tag{8.7.11}$$

Since code is set-valued, by Lemma 8.7.9 we may assume that $a = \overline{k}t$ for some $t : T$. Next, we can do induction on t. If $t \equiv \ell s$ for $s : S$, then we define (8.7.11) as in §8.7.1:

$$\begin{aligned}
(\dots, y_n, p_n, fks) &\mapsto (\dots, y_n, p_n, s, \mathsf{refl}_{gks}, gks), \\
(\dots, x_n, p_n, s, \mathsf{refl}_{fks}, fks) &\mapsto (\dots, x_n, p_n, gks).
\end{aligned}$$

These respect the equivalence relations, and define quasi-inverses just as before. Now suppose t varies along $m_{s,s'}(w)$ for some $w : ks = ks'$; we must show that (8.7.11) respects transporting along $\overline{k}mw$. By definition of $\overline{k}$, this essentially boils down to transporting along w itself. By the characterization of transport in path types, what we need to show is that

$$w_*(\dots, y_n, p_n, fks) = (\dots, y_n, p_n \centerdot fw, fks')$$

is mapped by (8.7.11) to

$$w_*(\dots, y_n, p_n, s, \mathsf{refl}_{gks}, gks) = (\dots, y_n, p_n, s, \mathsf{refl}_{gks} \centerdot gw, gks')$$

But this follows directly from the new generators we have imposed on the set-quotient relation defining code.

- The other three requisite equivalences are defined similarly.
- Finally, since the commutativity (8.7.2) is a mere proposition, by (-1)-connectedness of k we may assume that $a = ks$ and $a' = ks'$, in which case it follows exactly as before.

Theorem 8.7.12 (van Kampen with a set of basepoints)**.** *For all $u, v : P$ there is an equivalence*

$$\Pi_1 P(u, v) \simeq \mathsf{code}(u, v).$$

with code *defined as in this section.*

Proof. Basically just like before. To show that decode respects the new generators of the quotient relation, we use the naturality of h. And to show that decode respects the equivalences such as (8.7.11), we need to induct on $\overline{k}$ and

on T in order to decompose those equivalences into their definitions, but then it becomes again simply functoriality of f and g. The rest is easy. In particular, no additional argument is required for $\mathsf{encode} \circ \mathsf{decode}$, since the goal is to prove an equality in a set, and so the case of h is trivial. □